

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

<https://github.com/seejokim/na-python>

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업 혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
3	선형연립방정식 수치해법	3
4	수치미분 (Numerical Differentiation)	4
5	수치적분 (Numerical Integration)	5
6	수치 보간 (Interpolation)	6
7	수치 회귀 (Numerical Regression)	7
8	비선형연립방정식 수치해법	8
9	상미분 방정식 (초기치문제)	9
9.1	초기치문제 상미분 방정식 수치해석 개요	9
9.1.1	초기치문제 상미분 방정식 수치해석 개요	9
9.1.2	1계 상미분방정식과 연립방정식의 개념	9
9.2	Euler Method	10
9.2.1	Euler Method 개요	10
9.2.2	Explicit Euler Method	10
9.2.3	1계 연립 ODE에 대한 Explicit Euler Method	11
9.2.4	Implicit Euler Method	12
9.2.5	1계 연립 ODE에 대한 Implicit Euler Method	12
9.3	2차 Runge-Kutta Method (RK2)	13
9.4	4차 Runge-Kutta Method (RK4)	13
9.5	고계 상미분방정식을 1계 연립 ODE로 변환 방법	13
9.5.1	Mass-Spring-Damper계의 1계 연립 ODE 변환	13
9.5.2	파이썬을 이용한 RK4 프로그래밍	15
9.6	미분-적분 방정식 수치기법	15
9.6.1	Differential-Integral Equation Solution (Euler, RK2, RK4)	16
9.7	연습문제	16

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

Chapter 3

선형연립방정식 수치해법

Chapter 4

수치미분 (Numerical Differentiation)

Chapter 5

수치적분 (Numerical Integration)

Chapter 6

수치 보간 (Interpolation)

Chapter 7

수치 회귀 (Numerical Regression)

Chapter 8

비선형연립방정식 수치해법

Chapter 9

상미분 방정식 (초기치문제)

9.1 초기치문제 상미분 방정식 수치해석 개요

9.1.1 초기치문제 상미분 방정식 수치해석 개요

일반적인 1차 ODE:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

이를 초기치 문제(IVP)라고 한다.

9.1.2 1계 상미분방정식과 연립방정식의 개념

1. 1계 상미분방정식(Initial Value Problem)

1계 상미분방정식은 다음과 같이 주어진다.

$$\frac{dx}{dt} = f(x, t), \quad x(t_0) = x_0 \quad (9.1.2-1)$$

이는 초기조건 $x(t_0) = x_0$ 를 갖는 초기값 문제(IVP)이다.
수치해석에서는 미분을 차분으로 근사한다.

$$\frac{dx}{dt} \approx \frac{x^{n+1} - x^n}{\Delta t} \quad (9.1.2-2)$$

여기서

x^n : 현재값, x^{n+1} : 미래값, Δt : 시간 간격

대표적 방법:

- Explicit Euler Method
- Implicit Euler Method
- Runge-Kutta Method

—

2. 연립 1계 상미분방정식

미지수가 여러 개인 경우 다음과 같이 벡터 형태로 표현한다.

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

연립 방정식은

$$\frac{dx_1}{dt} = f_1(x_1, x_2, \dots, x_n, t) \quad (9.1)$$

$$\frac{dx_2}{dt} = f_2(x_1, x_2, \dots, x_n, t) \quad (9.2)$$

$$\frac{dx_n}{dt} = f_n(x_1, x_2, \dots, x_n, t) \quad (9.1.2-3)$$

또는 벡터 형태로

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

로 표현한다.

이러한 시스템은 동적 시스템 해석, 물리 모델링, 공학 계산 등에 활용된다.

9.2 Euler Method

9.2.1 Euler Method 개요

테일러 전개 1차 근사:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

9.2.2 Explicit Euler Method

1. 원리

Explicit Euler 방법은 가장 단순한 시간 적분 방법이다. 정확도는 1차이며, 전역 오차는

$$O(\Delta t)$$

이다.

미분을 전진 차분으로 근사하면

$$\frac{dx}{dt} = f(x, t) \Rightarrow \frac{x^{n+1} - x^n}{\Delta t} = f(x^n, t^n) \quad (9.2.2-1)$$

2. 계산식

위 식을 x^{n+1} 에 대해 정리하면

$$x^{n+1} = x^n + \Delta t f(x^n, t^n) \quad (9.2.2-2)$$

즉, 현재 시점의 기울기를 이용하여 다음 값을 계산하는 방법이다.

3. 연립 상미분방정식에 대한 형태

벡터 형태의 방정식

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

에 대해 Explicit Euler는

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \Delta t \mathbf{f}(\mathbf{x}^n, t^n) \quad (9.2.2-3)$$

로 표현된다.

특징

- 구현이 매우 간단하다.
- 정확도는 낮다 (1차 정확도).
- 시간 간격이 크면 불안정해질 수 있다.

```
def explicit_euler(f, t0, y0, h, n):
    t = t0
    y = y0
    for _ in range(n):
        y = y + h*f(t, y)
        t = t + h
    return y
```

9.2.3 1계 연립 ODE에 대한 Explicit Euler Method

1. 기본 형태

연립 상미분방정식은 일반적으로 다음과 같이 표현된다.

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (9.2.3-1)$$

2. 성분별 표현

예를 들어 N 개의 미지수를 가지는 경우,

$$\frac{dx_1}{dt} = f_1(x_1, x_2, \dots, x_N, t), \quad \frac{dx_2}{dt} = f_2(x_1, x_2, \dots, x_N, t), \quad \dots \quad \frac{dx_N}{dt} = f_N(x_1, x_2, \dots, x_N, t) \quad (9.2.3-2)$$

3. Explicit Euler 적용

Explicit Euler 방법을 적용하면 각 성분에 대해

$$\frac{x_i^{n+1} - x_i^n}{\Delta t} = f_i(x_1^n, x_2^n, \dots, x_N^n, t^n), \quad i = 1, 2, \dots, N \quad (9.2.3-3)$$

따라서 갱신식은

$$x_i^{n+1} = x_i^n + \Delta t f_i(x_1^n, x_2^n, \dots, x_N^n, t^n) \quad (9.3)$$

이다.

벡터 형태

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \Delta t \mathbf{f}(\mathbf{x}^n, t^n) \quad (9.4)$$

```
def euler_system(F, t0, y0, h, n):
    t = t0
    y = y0
    for _ in range(n):
        y = y + h*F(t,y)
        t += h
    return y
```

9.2.4 Implicit Euler Method

암시적 공식:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

9.2.5 1계 연립 ODE에 대한 Implicit Euler Method

1. 기본 형태

연립 상미분방정식은 일반적으로 다음과 같이 표현된다.

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (9.2.5-1)$$

2. 성분별 표현

예를 들어 N 개의 미지수를 가지는 경우,

$$\frac{dx_1}{dt} = f_1(x_1, x_2, \dots, t), \quad \text{quad} \quad \frac{dx_2}{dt} = f_2(x_1, x_2, \dots, t), \quad \dots \quad \frac{dx_N}{dt} = f_N(x_1, x_2, \dots, t) \quad (9.2.5-2)$$

3. Implicit Euler 적용

Implicit Euler 방법은 다음 시점 $(n+1)$ 의 값을 사용한다.

$$\frac{x_i^{n+1} - x_i^n}{\Delta t} = f_i(x_1^{n+1}, x_2^{n+1}, \dots, x_N^{n+1}, t^{n+1}), \quad i = 1, \dots, N \quad (9.2.5-3)$$

따라서 갱신식은

$$x_i^{n+1} = x_i^n + \Delta t f_i(x_1^{n+1}, x_2^{n+1}, \dots, x_N^{n+1}, t^{n+1}) \quad (9.5)$$

이다.

벡터 형태

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \Delta t \mathbf{f}(\mathbf{x}^{n+1}, t^{n+1}) \quad (9.6)$$

※ 특징

Implicit Euler는 다음 시점의 미지수 $\mathbf{x}^{n+1}$ 이 식 내부에 포함되므로 일반적으로 비선형 방정식을 풀어야 한다. 따라서 Newton 방법 등 반복법을 이용하여 $\mathbf{x}^{n+1}$ 을 계산한다.

9.3 2차 Runge-Kutta Method (RK2)

일반형:

$$k_1 = f(t_n, y_n)$$

$$k_2 = f(t_n + \alpha h, y_n + \alpha h k_1)$$

$$y_{n+1} = y_n + h(\beta_1 k_1 + \beta_2 k_2)$$

9.4 4차 Runge-Kutta Method (RK4)

$$k_1 = f(t_n, y_n)$$

$$k_2 = f(t_n + h/2, y_n + h k_1/2)$$

$$k_3 = f(t_n + h/2, y_n + h k_2/2)$$

$$k_4 = f(t_n + h, y_n + h k_3)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

```
def rk4(f, t0, y0, h, n):
    t = t0
    y = y0
    for _ in range(n):
        k1 = f(t, y)
        k2 = f(t+h/2, y+h*k1/2)
        k3 = f(t+h/2, y+h*k2/2)
        k4 = f(t+h, y+h*k3)
        y = y + h*(k1+2*k2+2*k3+k4)/6
        t += h
    return y
```

9.5 고계 상미분방정식을 1계 연립 ODE로 변환 방법

9.5.1 Mass-Spring-Damper계의 1계 연립 ODE 변환

질량-스프링-댐퍼 시스템은 다음 2차 미분방정식으로 표현된다.

$$my'' + cy' + ky = f(t) \quad (9.5.1-1)$$

① 1계 연립 ODE로 변환 변수 치환:

$$y_1 = y, \quad y_2 = \frac{dy}{dt}$$

그러면

$$\frac{dy_1}{dt} = y_2 \quad (9.5.1-2)$$

원식을 정리하면

$$m \frac{dy_2}{dt} = -cy_2 - ky_1 + f(t) \quad (9.5.1-3)$$

따라서 1계 연립 시스템은

$$\frac{d}{dt} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m}f(t) \end{bmatrix}$$

즉,

$$\dot{\mathbf{y}} = A\mathbf{y} + \mathbf{g}(t)$$

—

② RK4 적용 1계 연립 ODE

$$\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$$

에 대해 RK4는

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (9.5.1-4)$$

여기서

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(t_n, \mathbf{y}_n) \\ \mathbf{k}_2 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_1\right) \\ \mathbf{k}_3 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_2\right) \\ \mathbf{k}_4 &= \mathbf{f}(t_n + h, \mathbf{y}_n + h\mathbf{k}_3) \end{aligned}$$

—

요약 핵심

- 2차 ODE $\rightarrow$ 1계 연립 ODE 변환
- 상태벡터: $\mathbf{y} = [y_1, y_2]^T$
- 행렬형: $\dot{\mathbf{y}} = A\mathbf{y} + \mathbf{g}(t)$
- 수치해법: RK4 적용

9.5.2 파이썬을 이용한 RK4 프로그래밍

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter09/section_09_05_02_rk4_programming.py

9.6 미분-적분 방정식 수치기법

주어진 문제는 다음과 같은 미분-적분 방정식이다.

$$\frac{dy}{dt} = -y + I(t) \quad (9.6-1)$$

여기서 적분항은

$$I(t) = \int_0^t e^{-\tau} d\tau \quad (9.6-2)$$

초기조건은

$$y(0) = 1$$

시간격자:

$$t_{n+1} = t_n + h$$

—

① Euler 방법

$$y_{n+1} = y_n + (-y_n + I_n)\Delta t \quad (9.6-3)$$

—

② RK2 (2차 Runge-Kutta)

$$k_1 = f(t_n, y_n, I_n) \quad (9.7)$$

$$k_2 = f(t_n + \Delta t, y_n + \Delta t k_1, I_{n+1}) \quad (9.8)$$

$$y_{n+1} = y_n + \frac{\Delta t}{2}(k_1 + k_2) \quad (9.6-4)$$

—

③ RK4 (4차 Runge-Kutta)

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (9.6-5)$$

$$k_1 = f(t_n, y_n, I_n) \quad (9.9)$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1, I_{n+\frac{1}{2}}\right) \quad (9.10)$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2, I_{n+\frac{1}{2}}\right) \quad (9.11)$$

$$k_4 = f(t_n + h, y_n + hk_3, I_{n+1}) \quad (9.6-6)$$

—
적분항의 수치 계산
적분 $I(t)$ 는 누적합으로 계산한다.

$$I_{n+1} = I_n + e^{-t_n} \Delta t \quad (9.6-7)$$

9.6.1 Differential-Integral Equation Solution (Euler, RK2, RK4)

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter09/section_09_06_01_differential_integral_equation.py

미분-적분 혼합 형태 문제도 수치적으로 Euler, RK 방법으로 접근 가능하다.

9.7 연습문제